SQL Case Study — CAC Channel Analysis

Tools: BigQuery (Google Cloud) · SQL (Window Functions, CTEs)

Dataset: Synthetic advertising data from TikTok, META, and Google ad accounts

Project: `expanded-dryad-495912-i9`

Overview

This case study analyzes the Customer Acquisition Cost (CAC) across three paid marketing channels — META, TikTok, and Google — using raw cumulative snapshot data from advertising dashboards. The key challenge was correctly handling cumulative data structure to avoid ~100x inflated spend figures.

Dataset

Table `marketing_ads_raw` — raw data from TikTok, META, and Google ad accounts.

> ⚠️ **Key characteristic:** data is cumulative — each row shows metrics accumulated from the beginning of the day up to the moment of `timestamp`. During a single day, one `ad_id` may have 3–6 snapshots.

Step 1 — Deduplication

Because of the cumulative structure, a simple `SUM(spend)` produces a result inflated by 2–3x. We need to keep only **one row per** `(ad_id, date)` — the most recent one by `timestamp`.

We use `ROW_NUMBER()` with a partition on `(ad_id, date)` ordered by `timestamp DESC`, then filter `WHERE rn = 1`.

```
WITH dedup AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY ad_id, date
      ORDER BY timestamp DESC
    ) AS rn
  FROM `expanded-dryad-495912-i9.hw.marketing_ads_raw`
)
SELECT *
FROM dedup
WHERE rn = 1
```

📊 Results: https://docs.google.com/spreadsheets/d/1RIIDif0PIbpYajsvgaQP_rhTZwidIPrYXXhWsMfdutk/edit?usp=sharing

Validation — checking for any remaining duplicates per `(ad_id, date)`:

```
WITH dedup AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY ad_id, date
      ORDER BY timestamp DESC
    ) AS rn
  FROM `expanded-dryad-495912-i9.hw.marketing_ads_raw`
),
deduped AS (
  SELECT *
  FROM dedup
```

```
  WHERE rn = 1
)
SELECT ad_id, date, COUNT(*) AS cnt
FROM deduped
GROUP BY 1, 2
HAVING cnt > 1
```

✅ Empty result — deduplication successful. **8,814 rows → 1,950 rows.**

Step 2 — Daily Metrics

First Attempt — Using SUM

After deduplication, we aggregated by `(source, date)` using a standard `SUM`:

```
WITH dedup AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY ad_id, date
      ORDER BY timestamp DESC
    ) AS rn
  FROM `expanded-dryad-495912-i9.hw.marketing_ads_raw`
),
deduped AS (
  SELECT *
  FROM dedup
  WHERE rn = 1
),
daily AS (
  SELECT
    source,
    date,
    SUM(spend)         AS daily_spend,
    SUM(impressions)   AS daily_impressions,
    SUM(clicks)        AS daily_clicks,
    SUM(installs)      AS daily_installs,
    SUM(registrations) AS daily_registrations
  FROM deduped
  GROUP BY source, date
)

SELECT *
FROM daily
ORDER BY source, date
```

📊 Results: https://docs.google.com/spreadsheets/d/1mYCLZeWQOvU6QZLn3Mu3cImmyz74_hAM2tpOcHap-sQ/edit?usp=sharing

> ⚠️ **Problem:** the results showed values growing day over day — the data is cumulative not only within a day, but also across days. That is, `spend` on January 3rd already includes everything from January 2nd. A simple `SUM` again produces an incorrect result.

Fix — Using LAG

To get the real daily increment, we calculate the difference between consecutive days using `LAG`. We first compute the increment for each `ad_id` individually, then aggregate by channel and date.

```
WITH dedup AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY ad_id, date
      ORDER BY timestamp DESC
    ) AS rn
  FROM `expanded-dryad-495912-i9.hw.marketing_ads_raw`
),
deduped AS (
  SELECT * FROM dedup
  WHERE rn = 1
),
daily_per_ad AS (
  SELECT
    source, date, ad_id,
    spend         - LAG(spend)         OVER w AS daily_spend,
    impressions   - LAG(impressions)   OVER w AS daily_impressions,
    clicks        - LAG(clicks)        OVER w AS daily_clicks,
    installs      - LAG(installs)      OVER w AS daily_installs,
    registrations - LAG(registrations) OVER w AS daily_registrations
  FROM deduped
  WINDOW w AS (PARTITION BY ad_id ORDER BY date)
),
daily AS (
  SELECT
    source,
    date,
    SUM(daily_spend)         AS daily_spend,
    SUM(daily_impressions)   AS daily_impressions,
    SUM(daily_clicks)        AS daily_clicks,
    SUM(daily_installs)      AS daily_installs,
    SUM(daily_registrations) AS daily_registrations
  FROM daily_per_ad
  WHERE daily_spend IS NOT NULL -- first day of each ad_id has no previous row → NULL
  GROUP BY source, date
)

SELECT *
FROM daily
ORDER BY source, date
```

📊 Results: https://docs.google.com/spreadsheets/d/1DoL5digxG4xbYJQNCEfnGrIMyb-fesk9xjRyaEeKfTg/edit?usp=sharing

Step 3 — Summary Metrics by Channel

Three approaches were explored to arrive at the final channel-level metrics.

Attempt 1 — LAG Without Preserving the First Day

First version: daily increment via LAG, then two-level aggregation — first by `(source, date)`, then by `source`. However, the first day of each `ad_id` has no previous row — LAG returns NULL and the row is dropped by the filter.

```
WITH dedup AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY ad_id, date
      ORDER BY timestamp DESC
    ) AS rn
  FROM `expanded-dryad-495912-i9.hw.marketing_ads_raw`
),
deduped AS (
  SELECT * FROM dedup
  WHERE rn = 1
),
daily_per_ad AS (
  SELECT
    source, date, ad_id,
    spend         - LAG(spend)         OVER w AS daily_spend,
    impressions   - LAG(impressions)   OVER w AS daily_impressions,
    clicks        - LAG(clicks)        OVER w AS daily_clicks,
    installs      - LAG(installs)      OVER w AS daily_installs,
    registrations - LAG(registrations) OVER w AS daily_registrations
  FROM deduped
  WINDOW w AS (PARTITION BY ad_id ORDER BY date)
),
daily AS (
  SELECT
    source,
    date,
    SUM(daily_spend)         AS daily_spend,
    SUM(daily_impressions)   AS daily_impressions,
    SUM(daily_clicks)        AS daily_clicks,
    SUM(daily_installs)      AS daily_installs,
    SUM(daily_registrations) AS daily_registrations
  FROM daily_per_ad
  WHERE daily_spend IS NOT NULL
  GROUP BY source, date
)
SELECT
  source,
  ROUND(SUM(daily_spend), 2)                                                    AS total_spen
d,
  ROUND(SAFE_DIVIDE(SUM(daily_spend), SUM(daily_impressions)) * 1000, 2)        AS cpm,
  ROUND(SAFE_DIVIDE(SUM(daily_clicks) * 100.0, SUM(daily_impressions)), 2)      AS ctr_pct,
  ROUND(SAFE_DIVIDE(SUM(daily_installs) * 100.0, SUM(daily_clicks)), 2)         AS cr_click_in
stall_pct,
  ROUND(SAFE_DIVIDE(SUM(daily_registrations) * 100.0, SUM(daily_installs)), 2) AS cr_install_
reg_pct,
  ROUND(SAFE_DIVIDE(SUM(daily_spend), SUM(daily_registrations)), 2)             AS cac
FROM daily
GROUP BY source
ORDER BY cac ASC
```

> ⚠️ **Problem:** the result slightly differs from the expected benchmark — META comes out as ~$49,770 instead of ~$50,000. The reason: the first active day of each ad is lost because LAG has no previous value and returns NULL.

Attempt 2 — LAG with COALESCE (Preserving the First Day)

Fixed using `COALESCE`: if there is no previous day, we use the current value as-is, because the cumulative counter starts from zero — meaning the first snapshot equals the actual spend for that first day. The `WHERE daily_spend IS NOT NULL` filter is no longer needed.

```
WITH dedup AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY ad_id, date
      ORDER BY timestamp DESC
    ) AS rn
  FROM `expanded-dryad-495912-i9.hw.marketing_ads_raw`
),
deduped AS (
  SELECT * FROM dedup
  WHERE rn = 1
),
daily_per_ad AS (
  SELECT
    source, date, ad_id,
    COALESCE(spend         - LAG(spend)         OVER w, spend)         AS daily_spend,
    COALESCE(impressions   - LAG(impressions)   OVER w, impressions)   AS daily_impressions,
    COALESCE(clicks        - LAG(clicks)        OVER w, clicks)        AS daily_clicks,
    COALESCE(installs      - LAG(installs)      OVER w, installs)      AS daily_installs,
    COALESCE(registrations - LAG(registrations) OVER w, registrations) AS daily_registrations
  FROM deduped
  WINDOW w AS (PARTITION BY ad_id ORDER BY date)
),
daily AS (
  SELECT
    source,
    date,
    SUM(daily_spend)         AS daily_spend,
    SUM(daily_impressions)   AS daily_impressions,
    SUM(daily_clicks)        AS daily_clicks,
    SUM(daily_installs)      AS daily_installs,
    SUM(daily_registrations) AS daily_registrations
  FROM daily_per_ad
  GROUP BY source, date
)
SELECT
  source,
  ROUND(SUM(daily_spend), 2)                                                    AS total_spen
d,
  ROUND(SAFE_DIVIDE(SUM(daily_spend), SUM(daily_impressions)) * 1000, 2)        AS cpm,
  ROUND(SAFE_DIVIDE(SUM(daily_clicks) * 100.0, SUM(daily_impressions)), 2)      AS ctr_pct,
  ROUND(SAFE_DIVIDE(SUM(daily_installs) * 100.0, SUM(daily_clicks)), 2)         AS cr_click_in
stall_pct,
  ROUND(SAFE_DIVIDE(SUM(daily_registrations) * 100.0, SUM(daily_installs)), 2) AS cr_install_
reg_pct,
```

```
  ROUND(SAFE_DIVIDE(SUM(daily_spend), SUM(daily_registrations)), 2)            AS cac
FROM daily
GROUP BY source
ORDER BY cac ASC
```

Assert check: without LAG, a plain `SUM(spend)` would return META ≈ $5M (a ~100x overcount due to the cumulative nature of the data). After applying the daily increment with `COALESCE`, META = $50K — matching the expected benchmark, confirming the cumulative data was handled correctly.

Attempt 3 — Alternative Approach via Last Day (last_day)

For the final summary table, daily granularity is not strictly required — it is sufficient to take the **last day of each** `ad_id`, which already holds the cumulative totals for the entire campaign period. This approach is simpler and does not lose the first day as LAG does, but it does not provide an intermediate daily layer for trend analysis.

```
WITH dedup AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY ad_id, date
      ORDER BY timestamp DESC
    ) AS rn
  FROM `expanded-dryad-495912-i9.hw.marketing_ads_raw`
),
deduped AS (
  SELECT * FROM dedup
  WHERE rn = 1
),
-- take the last day for each ad_id — it holds the final cumulative value
last_day AS (
  SELECT *
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (
        PARTITION BY ad_id
        ORDER BY date DESC
      ) AS day_rn
    FROM deduped
  )
  WHERE day_rn = 1
)
SELECT
  source,
  ROUND(SUM(spend), 2)                                                  AS total_spend,
  ROUND(SAFE_DIVIDE(SUM(spend), SUM(impressions)) * 1000, 2)       AS cpm,
  ROUND(SAFE_DIVIDE(SUM(clicks) * 100.0, SUM(impressions)), 2)     AS ctr_pct,
  ROUND(SAFE_DIVIDE(SUM(installs) * 100.0, SUM(clicks)), 2)        AS cr_click_install_pct,
  ROUND(SAFE_DIVIDE(SUM(registrations) * 100.0, SUM(installs)), 2) AS cr_install_reg_pct,
  ROUND(SAFE_DIVIDE(SUM(spend), SUM(registrations)), 2)            AS cac
FROM last_day
GROUP BY source
ORDER BY cac ASC
```

📊 Results: https://docs.google.com/spreadsheets/d/1HCmeFxyI2CetLRa3hvdK8L69qRVPMy06Pp_kGKEjQ6M/edit?usp=sharing

✅ Validation: google ~$15K, meta ~$50K, tiktok ~$15K — matches the expected benchmark.

Step 4 — LTV/CAC (Bonus 1)

LTV values from a previous workshop are connected via a separate `ltv_values` CTE and joined to the final table.

```sql
WITH dedup AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY ad_id, date
      ORDER BY timestamp DESC
    ) AS rn
  FROM `expanded-dryad-495912-i9.hw.marketing_ads_raw`
),
deduped AS (
  SELECT * FROM dedup
  WHERE rn = 1
),
last_day AS (
  SELECT *
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (
        PARTITION BY ad_id
        ORDER BY date DESC
      ) AS day_rn
    FROM deduped
  )
  WHERE day_rn = 1
),
ltv_values AS (
  SELECT 'tiktok' AS source, 8.50 AS ltv UNION ALL
  SELECT 'meta',             6.20        UNION ALL
  SELECT 'google',           12.40
),
final AS (
  SELECT
    source,
    ROUND(SUM(spend), 2)                                          AS total_spend,
    ROUND(SAFE_DIVIDE(SUM(spend), SUM(impressions)) * 1000, 2)    AS cpm,
    ROUND(SAFE_DIVIDE(SUM(clicks) * 100.0, SUM(impressions)), 2)  AS ctr_pct,
    ROUND(SAFE_DIVIDE(SUM(installs) * 100.0, SUM(clicks)), 2)     AS cr_click_install_pct,
    ROUND(SAFE_DIVIDE(SUM(registrations) * 100.0, SUM(installs)), 2) AS cr_install_reg_pct,
    ROUND(SAFE_DIVIDE(SUM(spend), SUM(registrations)), 2)         AS cac
  FROM last_day
  GROUP BY source
)
SELECT
  f.source,
  f.total_spend,
  f.cpm,
  f.ctr_pct,
  f.cr_click_install_pct,
  f.cr_install_reg_pct,
```

```
  f.cac,
  l.ltv,
  ROUND(SAFE_DIVIDE(l.ltv, f.cac), 2) AS ltv_cac
FROM final f
JOIN ltv_values l ON f.source = l.source
ORDER BY f.cac ASC
```

📊 Results: https://docs.google.com/spreadsheets/d/1UI2DZQ-kXpa9trV8iVQC2j5Qmi_5wHanjK4BJGwByA4/edit?usp=sharing

Step 5 — Monthly CAC (Bonus 2)

The logic mirrors the daily metrics approach via LAG, but applied across months: we take the last day of each month for each `ad_id`, then compute the difference between consecutive months.

```
WITH dedup AS (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY ad_id, date
      ORDER BY timestamp DESC
    ) AS rn
  FROM `expanded-dryad-495912-i9.hw.marketing_ads_raw`
),
deduped AS (
  SELECT * FROM dedup
  WHERE rn = 1
),
last_day_of_month AS (
  SELECT *
  FROM (
    SELECT *,
      FORMAT_DATE('%Y-%m', date) AS month,
      ROW_NUMBER() OVER (
        PARTITION BY ad_id, FORMAT_DATE('%Y-%m', date)
        ORDER BY date DESC
      ) AS month_rn
    FROM deduped
  )
  WHERE month_rn = 1
),
monthly AS (
  SELECT
    source,
    month,
    ad_id,
    spend         - LAG(spend)         OVER w AS monthly_spend,
    registrations - LAG(registrations) OVER w AS monthly_registrations
  FROM last_day_of_month
  WINDOW w AS (PARTITION BY ad_id ORDER BY month)
)
SELECT
  source,
  month,
  ROUND(SUM(monthly_spend), 2)                                    AS monthly_spend,
```

```
  SUM(monthly_registrations)                                          AS monthly_registr
ations,
  ROUND(SAFE_DIVIDE(SUM(monthly_spend), SUM(monthly_registrations)), 2)   AS monthly_cac
FROM monthly
WHERE monthly_spend IS NOT NULL
GROUP BY source, month
ORDER BY source, month
```

📊 Results: https://docs.google.com/spreadsheets/d/1791kJSxeB0_TqEBxqQnGYzo5_B_niLMyWX8M3MZ0n6U/edit?usp=sharing

Final Results Table

source	total_spend	cpm	ctr_pct	cr_click_install_pct	cr_install_reg_pct	cac	ltv
meta	$50,000	14.0	1.2%	39.99%	93.98%	$3.10	$6.20
tiktok	$15,000	22.0	1.5%	30.97%	87.97%	$5.38	$8.50
google	$15,000	40.0	0.8%	36.96%	95.94%	$14.11	$12.40

LTV values sourced from a previous workshop session

Analysis

1. Which channel has the lowest CAC?

META — $3.10 per registration. This is significantly cheaper than Google ($14.11) and almost twice as cheap as TikTok ($5.38).

With an equal budget of $50K, Google would yield approximately 3,543 registrations ($50,000 / $14.11), while META delivers 16,129 ($50,000 / $3.10) — **4.5x more**. This makes the case for scaling META economically straightforward.

2. Where are the biggest funnel losses?

Biggest absolute drop — Impressions → Clicks (CTR). This is the weakest stage across all three channels: only 0.8–1.5% of people click on the ad, meaning 98–99% of the audience is lost here. Google has the worst CTR at **0.8%**.

Biggest lever for post-click optimization — Clicks → Installs. TikTok has the worst conversion rate at **30.97%** out of every 10 people who clicked, only 3 install the app.

Strongest stage — Installs → Registrations (88–96%). Once users install the app, they almost always complete registration. This stage is healthy across all channels and is not a priority for optimization.

3. Are META's spending levels justified?

Yes. META spends 3.3x more than TikTok and Google combined, yet maintains the **lowest CAC at $3.10**. Every dollar spent on META acquires more registrations than on any other channel. The high spend is not a problem — it is the result of scaling the most efficient channel.

4. Which channel is most profitable? (LTV/CAC)

Channel	LTV	CAC	LTV/CAC	Conclusion
META	$6.20	$3.10	2.00	✅ Every $1 spent returns $2
TikTok	$8.50	$5.38	1.58	✅ Profitable, but less efficient
Google	$12.40	$14.11	0.88	❌ Unprofitable — spending more than earning

Despite having the highest LTV among the three channels, Google does not break even due to its very high CAC.

5. Did channel performance change over time?

CAC remained **stable** across all three channels throughout the campaign (February–July 2024), as confirmed by the monthly breakdown:

source	month	monthly_spend	monthly_registrations	monthly_cac
google	2024-02	2,412.56	172	14.03
google	2024-03	2,438.35	173	14.09
google	2024-04	2,270.43	161	14.10
google	2024-05	2,467.77	175	14.10
google	2024-06	2,280.44	162	14.08
google	2024-07	706.85	50	14.14
meta	2024-02	7,681.53	2,475	3.10
meta	2024-03	8,275.11	2,666	3.10
meta	2024-04	7,549.54	2,432	3.10
meta	2024-05	7,945.47	2,559	3.10
meta	2024-06	6,988.97	2,252	3.10
meta	2024-07	3,652.52	1,175	3.11
tiktok	2024-02	2,508.47	467	5.37
tiktok	2024-03	2,153.42	400	5.38
tiktok	2024-04	2,351.83	437	5.38
tiktok	2024-05	2,405.29	447	5.38
tiktok	2024-06	2,244.01	417	5.38
tiktok	2024-07	1,321.31	246	5.37

The absence of trend suggests no seasonality and no degradation in traffic quality — channels behave predictably. That said, CAC stable to the cent across 6 consecutive months is unusual for real advertising campaigns, where fluctuations from seasonality, audience fatigue, and platform algorithm changes are expected. This level of consistency is more characteristic of a synthetically generated dataset than a live market.

Conclusions & Recommendations

- 🟢 **Scale META budget** — lowest CAC ($3.10) and best LTV/CAC ratio (2.0x)
- 🟡 **Improve Click→Install CR for TikTok** — 30.97% is the weakest post-click conversion across all channels; there is clear room for improvement
- 🔴 **Reconsider Google strategy** — LTV/CAC = 0.88, the channel is currently unprofitable